In [ ]:
```
import marimo as mo

import os

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from nilearn.image import load_img, math_img
from nilearn.glm.second_level import SecondLevelModel, non_parametric_infere
from nilearn.glm import threshold_stats_img
from nilearn.reporting import get_clusters_table

from nilearn.plotting import plot_img_on_surf, plot_stat_map, view_img

import nilearn
print(f"nilearn version: {nilearn.__version__}")
```

```
nilearn version: 0.13.1
```

In [ ]:
```
N_SUBS = 13
CONTRAST = "seed37_connectivity"
```

In [ ]:
```
current_dir = os.getcwd()
save_dir = f"{current_dir}/output_level2"
os.makedirs(save_dir, exist_ok=True)

beta_map_dir = f"{current_dir}/output"

subs = [str(i).zfill(2) for i in range(1, 16)]
subs.remove("08")
subs.remove("11")
assert len(subs) == N_SUBS
```

In [ ]:
```
beta_maps = []
for sub in subs:
    beta_maps.append(load_img(f"{beta_map_dir}/sub-{sub}/sub-{sub}_{CONTRAST

print(f"Loaded beta maps for {len(beta_maps)} subjects.")
```

```
Loaded beta maps for 13 subjects.
```

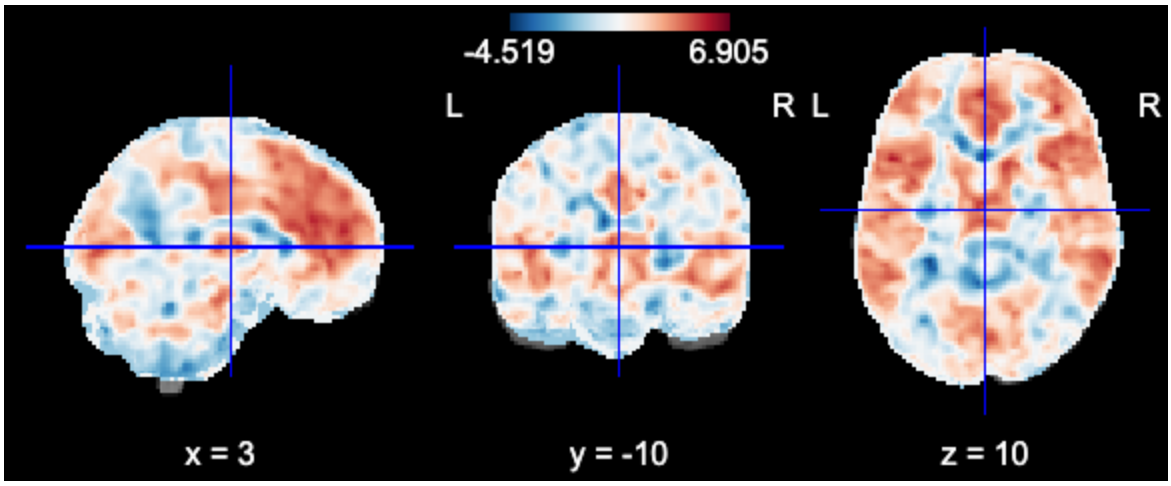
In [ ]:
```
# make sure all beta maps same shape and affine
shape1, affine1 = beta_maps[0].shape, beta_maps[0].affine
for i, bm in enumerate(beta_maps):
    if bm.shape != shape1 or not np.allclose(bm.affine, affine1):
        print(f"Warning: beta map for sub-{subs[i]} has different shape or a
        print(f"  shape: {bm.shape} vs {shape1}")
        print(f"  affine:\n{bm.affine}\nvs\n{affine1}")
```

In [ ]:
```
design_matrix = pd.DataFrame([1] * len(beta_maps), columns=["intercept"])

second_level_model = SecondLevelModel(smoothing_fwhm=None, n_jobs=-1)
second_level_model.fit(
```

```
    beta_maps,
    design_matrix=design_matrix,
)
zmap = second_level_model.compute_contrast(
    second_level_contrast="intercept",
    output_type="z_score",
)
zmap.to_filename(f"{save_dir}/group_{CONTRAST}_zmap.nii.gz")
```

In [ ]:
```
view_img(zmap, symmetric_cmap=False)
```

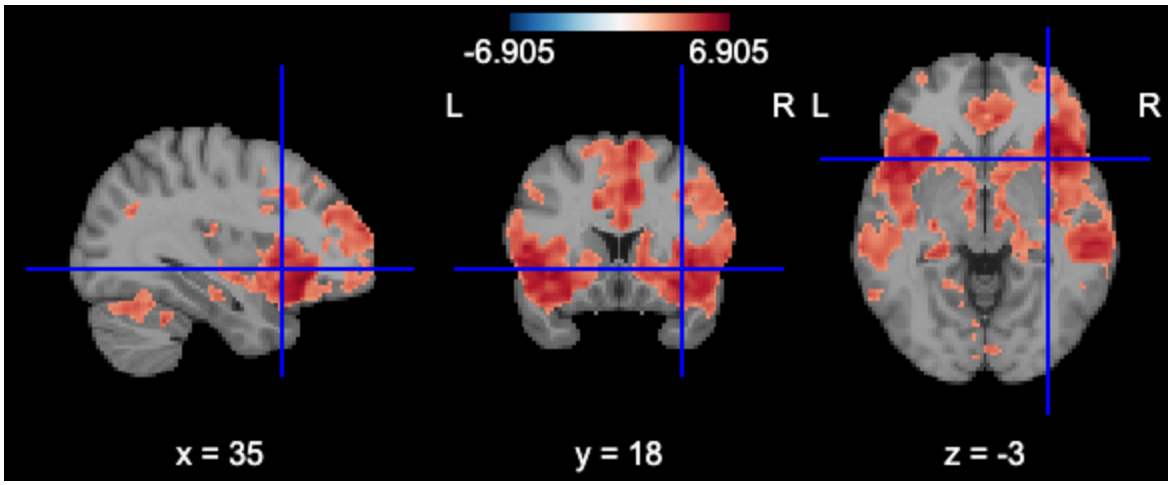
```
/Users/gabe/.cache/uv/archive-v0/8YAzRhKblqCcgmARF8Bv9/lib/python3.11/site-p
ackages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' wi
ll ignore the 'mask' of the MaskedArray.
  a.partition(kth, axis=axis, kind=kind, order=order)
```



Threshold

In [ ]:
```
thresholded_map, threshold = threshold_stats_img(
    zmap,
    alpha=0.01,                 # p < 0.05
    height_control=None,        # voxel-level control - if None, use thresho
    cluster_threshold=30,       # min cluster size - arbitrary... since GRFT
    two_sided=True,            # one-sided test (z>2.3)
    threshold=3.09               # voxel-wise z threshold
)
view_img(thresholded_map)
```

```
/Users/gabe/.cache/uv/archive-v0/8YAzRhKblqCcgmARF8Bv9/lib/python3.11/site-p
ackages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' wi
ll ignore the 'mask' of the MaskedArray.
  a.partition(kth, axis=axis, kind=kind, order=order)
```



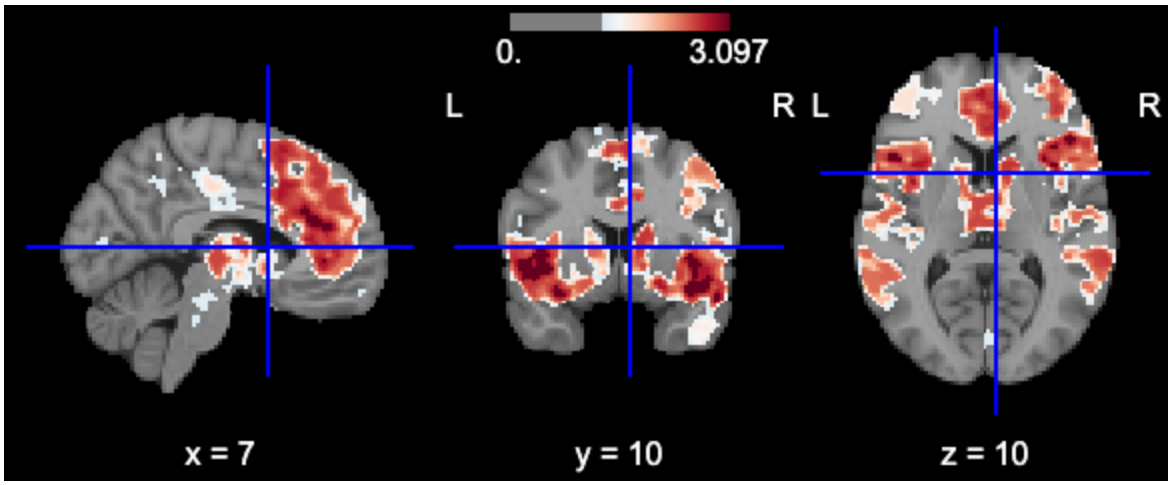
TFCE

In [ ]:
```
# # run TFCE
# out_dict = non_parametric_inference(
#     second_level_input=beta_maps,          # list of Nifti1Image or paths
#     design_matrix=design_matrix,
#     model_intercept=True,                  # one-sample test
#     n_perm=5000,
#     two_sided_test=True,                   # set False for directional hyp
#     tfce=True,                             # ← enables TFCE
#     threshold=None,                        # not needed when tfce=True
#     smoothing_fwhm=None,                   # smooth here if not done at 1s
#     n_jobs=6,                             # parallelise across all availab
#     verbose=1,
#     random_state=42,
# )
```

In [ ]:
```
# t_map        = out_dict["t"]
# tfce_scores  = out_dict["tfce"]
# logp_t       = out_dict["logp_max_t"]        # voxelwise FWE
# logp_tfce    = out_dict["logp_max_tfce"]     # TFCE-FWE  ← threshold at lo

# # save all maps
# t_map.to_filename(f"{save_dir}/group_{CONTRAST}_tmap.nii.gz")
# tfce_scores.to_filename(f"{save_dir}/group_{CONTRAST}_tfce.nii.gz")
# logp_t.to_filename(f"{save_dir}/group_{CONTRAST}_logp_t.nii.gz")
# logp_tfce.to_filename(f"{save_dir}/group_{CONTRAST}_logp_tfce.nii.gz")
```

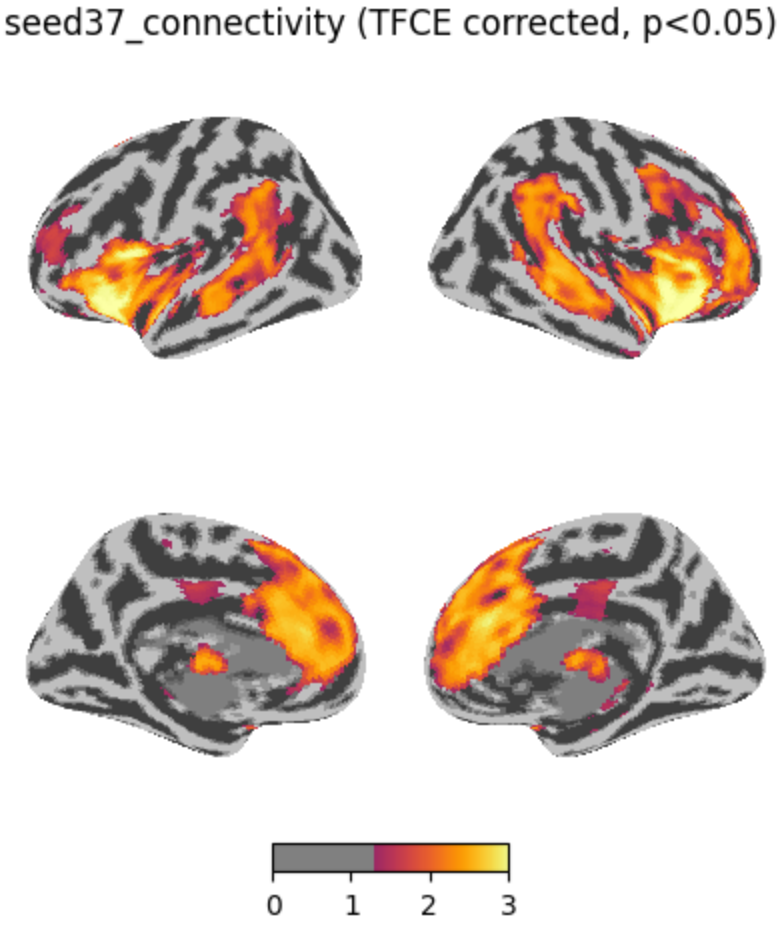
In [ ]:
```
# load and view TFCE map
t_map = load_img(f"{save_dir}/group_{CONTRAST}_tmap.nii.gz")
tfce_scores = load_img(f"{save_dir}/group_{CONTRAST}_tfce.nii.gz")
logp_tfce = load_img(f"{save_dir}/group_{CONTRAST}_logp_tfce.nii.gz")

THRESHOLD =-np.log10(0.05) # 1.3
view_img(logp_tfce, symmetric_cmap=False, threshold=THRESHOLD)
```

```
/Users/gabe/.cache/uv/archive-v0/8YAzRhKblqCcgmARF8Bv9/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' will ignore the 'mask' of the MaskedArray.
  a.partition(kth, axis=axis, kind=kind, order=order)
```



In [ ]:
```python
# surface plot
plot_img_on_surf(
    logp_tfce,
    surf_mesh="fsaverage5",
    views=["lateral", "medial"],
    hemispheres=["left", "right"],
    colorbar=True,
    cmap='inferno',
    symmetric_cmap=False,
    threshold=float(THRESHOLD),
    inflate=True,
    bg_on_data=False,
    title=f"{CONTRAST} (TFCE corrected, p<0.05)"
)
plt.savefig(f"{save_dir}/group_{CONTRAST}_tfce_surface.png", dpi=300)
plt.show()
```

```
/var/folders/44/klw90pzs3fz633mk4sk_j1hc0000gn/T/marimo_2306/__marimo__cell_nWHF_.py:2: UserWarning: 'symmetric_cmap' is not implemented for the matplotlib engine.
Got 'symmetric_cmap = False'.
Use 'symmetric_cmap = None' to silence this warning.
  plot_img_on_surf(
/var/folders/44/klw90pzs3fz633mk4sk_j1hc0000gn/T/marimo_2306/__marimo__cell_nWHF_.py:2: UserWarning: You provided a non integer threshold but configured the colorbar to use integer formatting.
  plot_img_on_surf(
```



Writeup

I first ran the connectivity analysis for each subject using the GLM-basde approach discussed in class. I smoothed the fMRI data (FWHM=6), and then used the `load_confounds` function to load the appropriate nuisance regressors: full motion (24 params), wm + csf, and compcor components. I also included linear and quadriatic drift trends, as well as global and local spike regressors.

I then extracted and z-scored the timeseries for the anterior insula mask (seed37 from the k50 parcellation mask). This time series and all the nuisance regressors mentioned above were combined into a design matrix for each run. We got the seed37 contrast (controlling for the covariates) for each subject, giving us which voxels coactivate with the signal in the anterior insula.

I then ran the a 2nd level model on these beta maps to get a group-level zmap. This map had lots of activation at different thresholds, so I decided to also run a stricter TFCE permutation correction. Here is the resulting surface plot:

As expected, we find significant coactivation in the bilateral anterior insula. We also see coactivation in the bilateral STS ant TPJ, as well as the mPFC, cingulate cortex, caudate nucleus, and thalamus.